DISEÑO DIGITAL 1.

TÍTULO DE LA ACTIVIDAD: **Modelado y simulación de un contador**	**CÓDIGO:** **AIP7**

FECHA:			
NOMBRE:		APELLIDOS:	

MODALIDAD:	Individual. Tutorial.	TIPO:	Presencial	DURACIÓN:	50 minutos

CALENDARIO:	Sesión presencial S7.	REQUISITOS:	Conocimientos adquiridos en Electrónica 2. Modelado y test VHDL.

CRITERIO DE ÉXITO:	

COMENTARIOS E INCIDENCIAS:

TIEMPO DEDICADO:	minutos	AUTOEVALUACIÓN: [entre 0 y 10 puntos]	No procede

Introducción

Esta actividad está dividida en dos partes. En la primera parte se presenta el modelo de un contador de 2 bits realizado con sentencias VHDL ya conocidas por el estudiante; en base a este ejemplo se relaciona la estructura del código VHDL que modela un sistema secuencial simple con la arquitectura del sistema modelado. En la segunda parte se muestran las limitaciones de esta estrategia de modelado, se presenta una fórmula aritmética de aplicación general para el modelado de contadores y se explican los recursos del lenguaje que dan soporte a la realización de modelos con operaciones aritméticas.

PARTE I. Modelado de contadores

Los contadores, al igual que los registros, son sistemas secuenciales síncronos en los que la salida es el estado de la memoria. Como ya sabe, el funcionamiento de este tipo de sistemas puede modelarse en VHDL con un proceso contenido en un Cuerpo de Arquitectura.

Consideremos, por ejemplo, el modelo lógico de un contador de 2 bits como el representado en la figura 1.

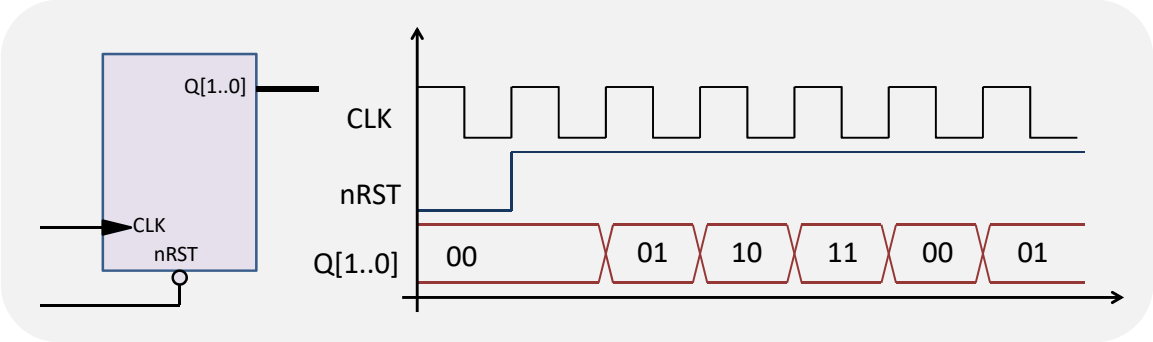


Figura 1.- Modelo lógico de un contador de 2 bits

Ya sabe que un proceso que modele el funcionamiento de este sistema debe ser sensible al reloj y a la entrada de inicialización asíncrona, debe disponer de una sentencia IF que priorice el funcionamiento asíncrono sobre el síncrono y debe especificar, mediante asignaciones, ambos tipos de funcionamiento.

```
process(clk, nRST)
begin
  if nRST = '0' then
    Q <= "00";

  elsif clk'event and clk = '1' then
    --Funcionamiento Sincrono

end process;
```

El funcionamiento síncrono de un contador tan sencillo como el de la figura 1 –un contador que sólo tiene cuatro estados de cuenta- puede describirse especificando detalladamente el estado al que evoluciona la cuenta (estado futuro), al ocurrir un flanco activo de reloj, en función del estado de cuenta anterior al flanco (estado actual). Esta estrategia para la especificación de la secuencia de cuenta puede materializarse fácilmente utilizando sentencias CASE o IF (figura 2).

```
case Q is
  when "00" =>
    Q <= "01";

  when "01" =>
    Q <= "10";

  when "10" =>
    Q <= "11";

  when "11" =>
    Q <= "00";

  when others =>
    null;
end case;
```

```
if Q = "00" then
  Q <= "01";

elsif Q = "01" then
  Q <= "10";

elsif Q = "10" then
  Q <= "11";

elsif Q = "11" then
  Q <= "00";

end if;
```

Figura 2.- Especificación de las transiciones entre estados de cuenta

Usando la versión realizada con la sentencia CASE, el código completo del proceso quedaría tal y como se muestra en la figura 3.

```
process(clk, nRST)
begin
  if nRST = '0' then
    Q <= "00";

  elsif clk'event and clk = '1' then
    case Q is
      when "00" =>
        Q <= "01";

      when "01" =>
        Q <= "10";

      when "10" =>
        Q <= "11";

      when "11" =>
        Q <= "00";

      when others =>
        null;

    end case;
  end if;
end process;
```

Figura 3.- Modelado del funcionamiento del contador

Para entender el modelo es imprescindible tener en cuenta que Q puede representar tanto el valor actual como el valor futuro de cuenta:

- El valor de Q que se evalúa en la sentencia CASE cuando se ejecuta el proceso por la ocurrencia de un flanco activo de reloj es el **valor actual** de Q, es decir, el valor de Q antes de la ocurrencia del flanco (Q(t)).

- El valor que se asigna (proyecta) a Q en las ramas de la sentencia CASE es el que tomará después del flanco, es decir, el **valor futuro** de Q (Q(t+1)).

En definitiva, en la sección que describe el funcionamiento síncrono, cuando Q está a la izquierda en una sentencia de asignación representa el estado de memoria (o cuenta) futuro y, cuando se evalúa su valor, el estado de memoria (o cuenta) actual.

Esto pone de manifiesto que el código de la sección que describe el funcionamiento síncrono del contador define la tabla de verdad de la lógica combinacional para el cálculo de transiciones de estados del contador –la tabla de verdad de un incrementador de 2 bits (Figura 4).

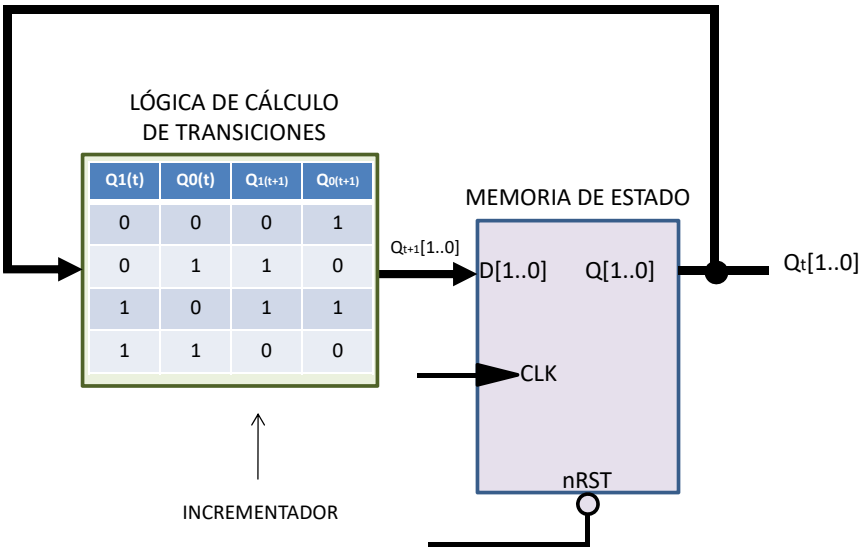


Figura 4.- Arquitectura del Contador de 2 bits

La figura 4 ilustra el hecho de que un contador es, efectivamente, un sistema secuencial con arquitectura de Moore en el que la salida es el estado de memoria. Esta arquitectura define, además, la complejidad máxima de los sistemas secuenciales que es posible modelar con un solo proceso VHDL –como veremos más adelante, la realización del modelo de sistemas secuenciales con arquitecturas más complejas requiere el empleo de dos o más procesos.

Un último comentario sobre el código del proceso que modela el contador: la última rama de la sentencia CASE, que comprende todas las combinaciones de dos bits que contienen valores metalógicos, ejecuta una sentencia VHDL que no se había utilizado hasta ahora, la sentencia NULL. Como su nombre indica, su ejecución no produce ningún efecto[1].

Realice las siguientes operaciones:

1. Descargue, si no lo ha hecho todavía, los ficheros para la actividad.

   Compruebe que en el directorio de esta actividad existe un fichero con el modelo del contador (*cont2b.vhd*) y un TB (*cont2b_tb.vhd*) para probar dicho modelo.

2. En el directorio de la actividad, cree un proyecto, *Proy_AIP7*, y añádale los dos ficheros.

3. Compile los ficheros y realice una simulación, con una duración de 1 microsegundo, para verificar su funcionamiento.

4. Modifique el código del fichero *cont2b.vhd* para que modele el funcionamiento de un contador descendente. Compile el código y simúlelo empleando el TB *cont2b_tb*.

[1] En el modelo del contador podría haberse optado por utilizar, en lugar de la sentencia NULL, la asignación de una indeterminación: Q <= “XX”;.

PARTE II. El paquete *STD_LOGIC_UNSIGNED*

La estrategia de modelado presentada en el apartado anterior no es aplicable a contadores con un gran número de estados de cuenta (con un **módulo** grande), ya que resultaría inviable la codificación de sentencias CASE o IF con cientos, miles o millones de alternativas (una por estado de cuenta). Pero en un contador ascendente el estado futuro de cuenta puede expresarse de manera genérica con la fórmula:

Estado Futuro = Estado Actual + 1

Esta propiedad se puede utilizar para definir las transiciones entre estados de cuenta y construir así el modelo de cualquier contador, independientemente del módulo que tenga. Por ejemplo, el modelo del contador de 2 bits del apartado 1 de esta actividad se puede modelar con un proceso como el que se muestra en la figura 5.

```
process(clk, nRST)
begin
  if nRST = '0' then
    Q <= "00";

  elsif clk'event and clk = '1' then
    Q <= Q + 1;

  end if;
end process;
```

Figura 5.- Modelado con operación aritmética

La descripción del funcionamiento síncrono se realiza ahora con una sentencia en la que se indica que el **estado futuro de cuenta** (la Q a la que se asigna valor en la sentencia), tras el flanco activo de reloj, es el **estado actual de cuenta** (la Q evaluada en el flanco) más uno.

Observe que la representación en una tabla de la sentencia de asignación Q <= Q + 1, para una codificación de Q en binario natural con 2 bits, no es otra cosa que la tabla de verdad de la lógica de cálculo de transiciones de la figura 4. Por tanto, el código de la figura 5 no es más que una forma más rápida y concisa de expresar el funcionamiento del contador.

Realice las siguientes operaciones:

1. Sustituya, en el proceso que modela el funcionamiento del contador, en el fichero *cont2b.vhd*, la sentencia CASE por una sentencia de asignación, Q <= Q + 1, como la que se emplea en la figura 5.

2. Salve y compile el fichero.

   Observe que el compilador indica que hay un error en el fichero. Si inicia el proceso de depuración del error averiguará que la causa del error es que no está definida la operación suma (Figura 6) para objetos de tipo *std_logic_vector*.

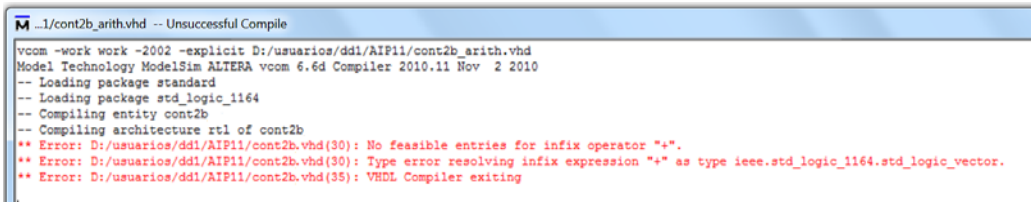


Figura 6.- Error de compilación

El motivo por el que el compilador no reconoce la suma como una operación que pueda aplicarse a objetos del tipo *std_logic_vector* es que en el paquete VHDL *std_logic_1164,* que es el único que resulta visible al código analizado, sólo se definen, para este tipo de datos[2], algunas operaciones lógicas básicas (AND, NAND, OR, NOR, XOR, XNOR y NOT).

Las operaciones aritméticas (suma, resta, producto y comparaciones de magnitud) para los tipos *std_logic* y *std_logic_vector* están definidas en otros paquetes:

1. En el paquete *std_logic_unsigned* se definen operaciones aritméticas que interpretan el valor de los objetos *std_logic* y *std_logic_vector* en binario natural.

2. En el paquete *std_logic_signed* se definen operaciones aritméticas que interpretan el valor de los objetos *std_logic* y *std_logic_vector* en complemento a dos.

Para poder emplear operaciones aritméticas en un modelo, debe añadirse una cláusula de visibilidad sobre uno de estos dos paquetes —sobre el paquete *std_logic_unsigned* cuando se desea operar en binario natural (es el que se usa con más frecuencia) o sobre el *std_logic_signed* cuando se desee hacerlo en complemento a dos.

Realice la siguiente operación:

1. Edite el fichero *cont2b.vhd* para añadir la cláusula de visibilidad que aparece en la línea 9 del código de la figura 7.

```
5
6   -- Cabecera: cláusulas de visibilidad
7   library ieee;
8   use ieee.std_logic_1164.all;
9   use ieee.std_logic_unsigned.all;
10
11  -- Entidad: unidad primaria VHDL que modela la interfaz
12  entity cont2b_a is
13  port(
14       clk:  in std_logic;
15       nRST: in std_logic;
16       Q:    buffer std_logic_vector(1 downto 0)
17       );
18  end entity;
```

Figura 7.- Cláusulas de visibilidad

2. Salve y compile el fichero. La compilación debe desarrollarse ahora con éxito.

[2] En este paquete se definen, además, otros tipos de datos y funciones de conversión entre tipos.

El lenguaje VHDL permite la *sobrecarga* de operaciones. Esto quiere decir que puede utilizarse el mismo nombre o identificador para operaciones que difieren en el tipo de datos de los operandos. Los paquetes aritméticos (*std_logic_unsigned* y *std_logic_signed*) aprovechan la sobrecarga de operaciones para definir diversas operaciones de suma y resta que se diferencian por el tipo de datos de los operandos. Ambas operaciones están definidas para los casos en que:

- Ambos operandos son del tipo *std_logic_vector*.

  Ejemplos:

  "0001" + "1110"
  A – "0100" siendo A un objeto de tipo *std_logic_vector*
  A + B siendo A y B objetos de tipo *std_logic_vector*

- Un operando es de tipo *std_logic_vector* y el otro es de tipo *std_logic*

  Ejemplos:

  "110001" + '1'
  A + "10110" siendo A un objeto de tipo *std_logic*
  A – B siendo uno de tipo *std_logic* y el otro de tipo *std_logic_vector*.

- Un operando es de tipo *std_logic_vector* y el otro es de tipo entero

  Ejemplos:

  "110001" - 7
  1 + A siendo A un objeto de tipo *std_logic_vector*
  A + B siendo uno de tipo entero y el otro de tipo *std_logic_vector*.

Cuando se compila el código se determina, en función del tipo de los operandos, que operación es la que corresponde asociar a cada caso. Si el compilador no encuentra una operación definida para los tipos de datos de los operandos, se genera un diagnóstico de error como el de la figura 6 -el modelo del contador se ha compilado con éxito, finalmente, porque la suma involucra a un objeto de tipo *std_logic_vector* (Q) y a una constante de tipo entero (1) y este caso está entre los definidos en el paquete *std_logic_unsigned*.

Realice las siguientes operaciones:

1. Realice una simulación para verificar el funcionamiento del nuevo modelo del contador.

Resumen

Para modelar contadores en VHDL:

1. Debe dar visibilidad al modelo sobre el paquete *std_logic_unsigned*; para ello debe añadir, a las cláusulas que preceden a la Declaración de Entidad -a las que venía utilizando hasta ahora-, la sentencia:

```
use ieee.std_logic_unsigned.all;
```

2. Debe usar las expresiones Q <= Q + 1 (ó Q <= Q – 1, si se trata de un contador descendente) para modelar la secuencia de cuenta del contador.